Generative AI Usage Log

Prompts and Interaction Log for ACM Project

Furkan Nar

May 28, 2026

Abstract

This document logs the prompts and interactions with generative AI assistants during the development and evaluation of the Affective Coherence Monitoring (ACM) project. Each entry includes the timestamp, the prompt content, and the corresponding AI output for transparency and reproducibility purposes.

Contents

Prompt Log

Prompt #1

Date: 28.05.2026
Time: 10:46 **(UTC+3)**
Context: Starting the conversation

Prompt

Hello Claude I'd request help from you regarding one of my preprints in order to create the code for it.

Output

Hello! I'd be happy to help you create code for your preprint. Please share the details — whether that's the paper itself, a description of the methods, algorithms, or analyses you need implemented, and any specific programming language or framework you have in mind.
What would you like to build?

Prompt #2

Date: 28.05.2026
Time: 10:48 **(UTC+3)**
Context: Sharing details regarding the preprint and my own progress

Prompt

Its about Affective Coherence Monitoring. My current progress: (I shared it in the original chat but it will be too big here) I have shared both the lexica.json, anchors.json and the current progress of the python file. I want you to make the json pipeline better and functional

Output

Looking at your preprint and current code, I have a few quick questions before diving in:

Prompt #3

Date: 28.05.2026
Time: 10:49 **(UTC+3)**
Context: QAs and boilerplate generation

Prompt

Q: What's the primary use case for the pipeline right now? A: Offline batch processing of text
Q: Which parts of the ACM pipeline do you want implemented in this pass? (Select all that apply) A: All of the above. I will write a critique soon about the generated code of yours.
Q: How should the keyword lexicon weights work? A: Per-emotion importance multipliers (as in the paper)

Output

Here's the full pipeline. Let me walk you through the key decisions:
**JSON loading (‘load$_{c}onfigs$‘)
‘EmotionalCentroidGenerator‘ — fixed a subtle bug in your original
‘KeywordScorer‘ — implements per-emotion importance multipliers as the paper specifies (§4 Step 1).
‘AffectiveCoherenceMonitor‘ — all 8 pipeline steps are in one class with clean separation.
‘ACMBatchProcessor‘ — the ‘reset$_{b}etween‘flagletsyouchoose$
A few things to look out for when you write your critique: The ‘"seeking"‘ key in your anchors JSON maps to ‘curiosity‘ via LEXICON$_{K}EY_{M}AP^{-}ifyourJSONuses"seeking"directly, you'llwanttoaddthatmapping.$

Prompt #4

Date: 28.05.2026
Time: 10:53 **(UTC+3)**
Context: Critique of the Pipeline and removing bloat code

Prompt

The way you are fetching the anchors and the lexica is wrong. It is supposed to be inside config/ another thing is that why would you even use this part "Keyword → emotion mapping (lexica.json key → ACM canonical name) $\text{LEXICON}_{K}EY_{M}AP : Dict[str, str] =$ $"happy" : "joy","sad" : "sadness","curious" : "curiosity","creative" : "creativity","fear" : "fear","an$

Output

Two clean fixes:
config/*.json — all JSON loading now points to the config/ sub-directory, both in build-acm-from-configs and the –main– block. $\text{LEXICON}_{K}EY_{M}APremovedentirely^{-}KeywordScorer. - -init - -nowiteratesemotion-wordsdirectly, skippinganykeynotinEMOTIONS(so"general"andanyfuturenon-emotionkeysaresilentlydroppedwithoutneedingatranslationlayer).Sameclean - upappliedinbuild - centroids.$

Prompt #5

Date: 28.05.2026
Time: 10:55 **(UTC+3)**
Context: Catching import and formatting errors

Prompt

You're using log. without doing "from logging import log" another fatal flaw is that the syntax of your code will almost entirely be unreachable due to wrong spaces, it also gives errors even from the beginning due to wrong formatting.

Output

Two concrete fixes applied throughout: Logging — removed the broken from logging import log entirely. Now it's logger = logging.getLogger(–name–) at module level, and every call is logger.info(...), logger.warning(...), etc. This is the standard Python pattern and will work correctly. Indentation/formatting — full rewrite with consistent 4-space indentation everywhere. The previous version had mixed indentation inside class methods and some if/else blocks that would have caused IndentationError or silently unreachable branches. Everything is now flat and unambiguous. The logic itself is unchanged — only the structural correctness was fixed. Let me know what else comes up in your critique.

Prompt #6

Date: 28.05.2026
Time: 10:58 **(UTC+3)**
Context: Adressing the forgotten equations from the original preprint

Prompt

It doesnt even contain all eqs.

Output

Here's what changed and why: Eq. 3 (centroid) — weighted sum now correctly uses raw emb (not normalised), with centrality computed from the normalised emb-n. The comment explicitly calls this out since it's a subtle distinction. Eqs. 4–5 (valence/arousal) — rewritten term-by-term to match the paper exactly rather than using a generic sum() over a dict. Each coefficient is explicit and traceable to the equation. Eqs. 8–16 (mismatch) — Ahigh, Alow, Stotal, and C are now all returned in the mismatch dict and surfaced in the output. Previously Ahigh/Alow were computed but discarded. Eqs. 20–21 (habituation) ordering fix — habituation is now called before -apply-decay, because Eq. 20 uses $E_c urrent, ewhichmustbethepre - decaystate. Previouslytheorderwasinverted, meaninghabituationwascomparingagainstthealready - updatedstate.$

Prompt #7

Date: 28.05.2026
Time: 11:07 **(UTC+3)**
Context: Better documentation of Results by finding the biggest argument (argmax)

Prompt

Also in sample texts show what emotion it classified as. For example you show many other statistics but dont show the classification, such as "I'm so happy today!" classified as Happy and etc.

Output

Two lines added to the output loop:
Classified as — takes the argmax over the habituated scores (post-habituation is the most meaningful final score after all pipeline stages) and prints the dominant emotion in uppercase with its score. All scores — prints every emotion's habituated score sorted descending, so you can see the full distribution at a glance, not just the winner. Useful for spotting mixed-emotion cases like the last sample text where sadness and joy may both be elevated.

Prompt #8

Date: 28.05.2026
Time: 11:11 **(UTC+3)**
Context: Improving the sample quality through well-established datasets

Prompt

It sometimes classifies the emotions wrongly or mixes it up so that means we need richer and better Anchors. I will use text from google-research-datasets/go$_e$*motions*

Output

That's a solid choice — GoEmotions is exactly the kind of richly annotated, large-scale corpus you need. It's actually cited in your paper as reference [5], so it's a natural fit.
A few things worth thinking through before you write the loading code:

Prompt #9

Date: 28.05.2026
Time: 11:12 **(UTC+3)**
Context: QAs and further implementation details

Prompt

Q: GoEmotions has 28 fine-grained emotion labels. How do you want to map them to your 8 ACM emotions? A: Let Claude propose a mapping based on the paper's valence/arousal definitions
Q: How many exemplar sentences per ACM emotion do you want to pull from GoEmotions for the centroids? A: 50 (balanced)

Output

The script does the following: Label index — iterates the full GoEmotions train split once and buckets every text by its labels. Since GoEmotions is multi-label, one text can appear in multiple buckets, which is handled cleanly. Deduplication — a shared used set ensures no sentence ends up as an exemplar for two different ACM emotions. Priority goes in the order emotions are defined in ACM-TO-GO, so if a text is tagged both curiosity and surprise in GoEmotions it only goes to whichever ACM emotion claims it first. Filtering — texts shorter than 15 or longer than 200 characters are dropped. Very short fragments hurt centroid quality; very long ones dilute the embedding signal toward a generic average. Output — writes directly to config/anchors.json in the same structure your existing pipeline expects ("emotional$_a$*nchors*" : ...), *sonochangesneededinthemainACMcode.Runitoncewithpythonbuild$_a$nchors.pyandthenewcentroidswi* $from - configs()$.

Prompt #10

Date: 28.05.2026
Time: 11:14 **(UTC+3)**
Context: Suggestion from previous code

Prompt

In the new code you generated you should import main.py (The previous code we made for ACM) in order for your newly code to work

Output

Two changes made:
Import — build-acm-from-configs and AffectiveCoherenceMonitor are now imported from main.py at the top, so the type annotation and factory call are both valid. Auto-rebuild — after writing the new anchors.json, the script immediately calls build-acm-from-configs("config/*.json") to recompute all centroids against the fresh GoEmotions exemplars. This means running python build-anchors.py is a single complete operation: fetch → write → rebuild.
Make sure build$_anchors.pyandmain.pysitinthesamedirectory, withconfig/asasubdirectoryofboth.$

Prompt #11

Date: 28.05.2026
Time: 11:17 **(UTC+3)**
Context: Questioning the dataset classification

Prompt

It seems that the dataset itself is flawed about curiosity: "[CURIOSITY] • "Muscles? You mean blood balloons?" • What does "I moved on her like a bitch" mean? I never understood that line • Goalpost shifting much? [CREATIVITY] • Why should I waste my time with that? • Is this an /r/antijokes ? • Isn't SOHR group of people(organisation)? lol dont tell me its just one guy."

Output

Here's what the two-pass filter does concretely: Pass 1 — encodes all 200 candidates for an emotion, computes a simple L2-normalised mean embedding as a rough centroid. This is intentionally naive — it just needs to be directionally correct, not perfect. Pass 2 — scores every candidate by cosine similarity to that rough centroid, then keeps only the top 50. "Blood balloons" and "Why should I waste my time" would score very low against a curiosity centroid and get dropped naturally, with no hardcoded rules needed. Transparency logging — for each emotion it logs the similarity range of kept vs dropped candidates, so you can see exactly how much filtering is happening. If the kept range is very low (e.g. below 0.3) it's a signal the GoEmotions label itself is too noisy and the mapping may need revisiting.
